

CHAPTER 08

영상의 기하학적 변환

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

8장 영상의 기하학적 변환

8.1 어파인 변환

8.2 투시 변환

8.1 어파인 변환

이동 변환(translation transformation)

- 영상을 가로 또는 세로 방향으로 일정 크기만큼 이동시키는 연산을 의미하며 시프트(shift) 연산이라고도 함

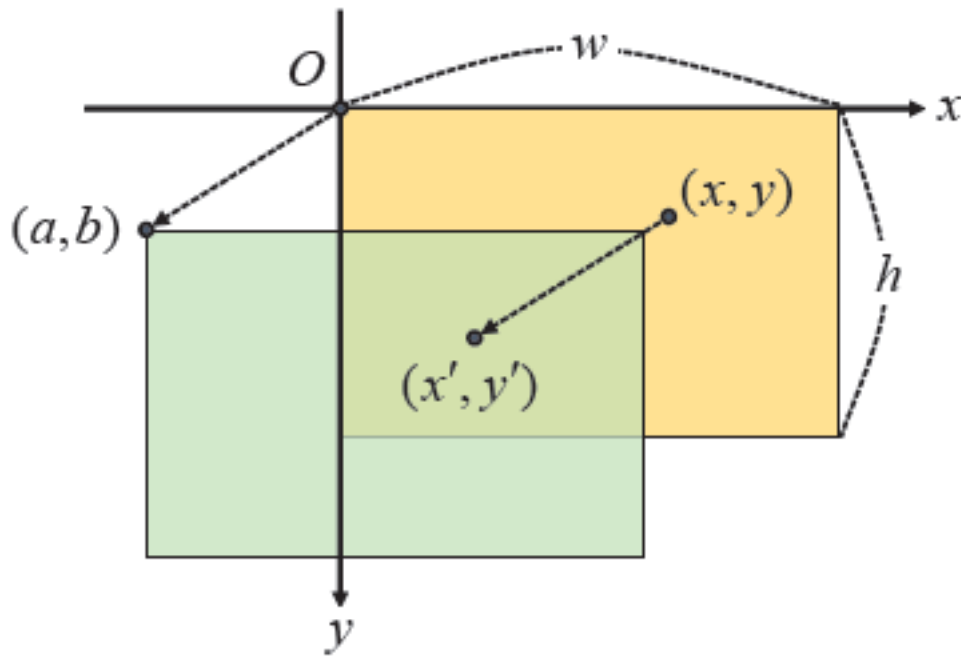


그림 8-4 영상의 이동 변환

이동 변환

- 입력 영상의 모든 좌표를 x 방향으로 a 만큼, y 방향으로 b 만큼 이동하는 변환을 수식으로 나타내면 다음과 같음

$$\begin{cases} x' = x + a \\ y' = y + b \end{cases}$$

- 앞 수식에서 (x, y) 는 입력 영상의 픽셀 좌표이고, (x', y') 는 결과 영상의 픽셀 좌표임
- 앞 수식을 행렬을 이용하면 다음과 같이 하나의 식으로 표현가능.

$$\begin{aligned} \begin{pmatrix} x' \\ y' \end{pmatrix} &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} a \\ b \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 & a \\ 0 & 1 & b \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \end{aligned}$$

이동 변환

- 영상을 x 방향으로 a만큼, y 방향으로 b만큼 이동하는 어파인 변환 행렬 M은 다음과 같음

$$M = \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \end{bmatrix}$$

- 영상을 이동변환하려면 앞과 같은 형태의 2×3 실수 행렬 M을 만들고, 이를 warpAffine() 함수 인자로 전달함

코드8-2: 이동 변환

- 코드 8-2는 tekapo.bmp 호수 영상을 가로로 150픽셀, 세로로 100픽셀만큼 이동시키고 그 결과를 화면에 출력함

코드8-2: 이동 변환

```
#include "opencv2/opencv.hpp"
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat src = imread("tekapo.bmp");
    if (src.empty()) {cerr << "Image load failed!" << endl; return -1;}

    Mat M = Mat_<double>({ 2, 3 }, { 1, 0, 150, 0, 1, 100 });
    //84~85페이지 참고할 것

    Mat dst;
    warpAffine(src, dst, M, Size());
    //double data[] = { 1, 0,150, 0, 1, 100 };
    //Mat M(2, 3, CV_64FC1, data);

    imshow("src", src);
    imshow("dst", dst);
    waitKey();
    return 0;
}
```


코드8-2: 이동 변환

- 결과 영상에서 호수 영상이 (150, 100) 좌표부터 나타나는 것을 확인할 수 있고,
- 입력 영상의 픽셀 값이 복사되지 않은 영역은 검은색으로 채워진 것을 확인할 수 있음

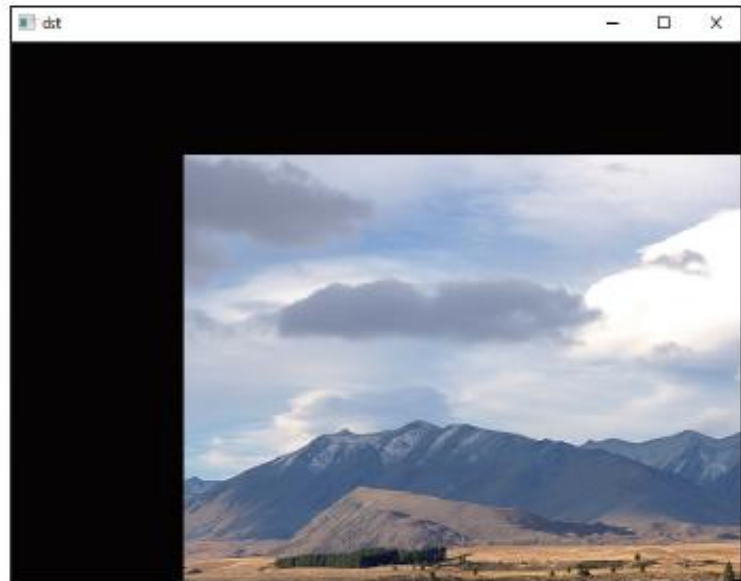


그림 8-5 영상의 이동 변환 결과

전단 변환

- 전단 변환(shear transformation)은 직사각형 형태의 영상을 한쪽 방향으로 밀어서 평행사변형 모양으로 변형되는 변환이며 층밀림 변환이라고도 함
- 전단 변환은 가로 방향 또는 세로 방향으로 각각 정의할 수 있음
- 전단 변환은 영상의 픽셀을 가로 방향 또는 세로 방향으로 이동하지만, 픽셀이 어느 위치에 있는가에 따라 이동 정도가 달라짐
- 입력 영상에서 원점은 전단 변환에 의해 이동하지 않고 그대로 원점에 머물러 있음

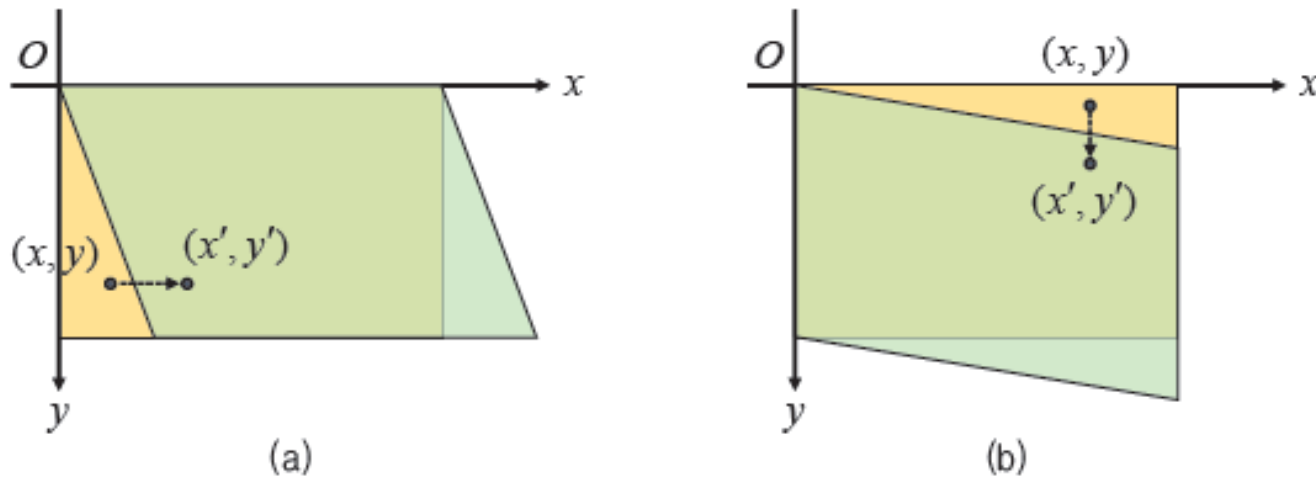


그림 8-6 영상의 전단 변환

전단 변환

- 그림 8-6(a)와 같이 y 좌표가 증가함에 따라 영상을 가로 방향으로 조금씩 밀어서 만드는 전단 변환 수식은 다음과 같음

$$\begin{cases} x' = x + m_x y \\ y' = y \end{cases} \quad \text{또는} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & m_x \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{pmatrix} 1 & m_x & 0 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- 그림 8-6(b)와 같이 x 좌표가 증가함에 따라 영상을 세로 방향으로 조금씩 밀어서 만드는 전단 변환 수식은 다음과 같음

$$\begin{cases} x' = x \\ y' = m_y x + y \end{cases} \quad \text{또는} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ m_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ m_y & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

전단 변환

- 앞의 두 수식에서 m_x 와 m_y 는 영상으로 각각 가로 방향과 세로 방향으로 밀림 정도를 나타내는 실수임
- 전단 변환을 나타내는 2×3 어파인 변환 행렬 M 은 다음과 같이 나타낼 수 있음

$$M = \begin{bmatrix} 1 & m_x & 0 \\ 0 & 1 & 0 \end{bmatrix} \text{ 또는 } M = \begin{bmatrix} 1 & 0 & 0 \\ m_y & 1 & 0 \end{bmatrix}$$

코드 8-3: 전단 변환

- 코드 8-3는 tekapo.bmp 호수 영상에 대해 y 좌표가 증가함에 따라 $0.3y$ 에 해당하는 x 좌표에서 원본 영상 픽셀이 나타나기 시작하는 전단 변환을 수행함

코드8-3: 전단 변환

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat src = imread("tekapo.bmp");
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1; }

    double mx = 0.3;
    Mat M = Mat_<double>({ 2, 3 }, { 1, mx, 0, 0, 1, 0 }); //84~85페이지참고
    Mat dst;
    warpAffine(src, dst, M, Size(cvRound(src.cols + src.rows * mx), src.rows));

    imshow("src", src);
    imshow("dst", dst);
    waitKey();
    return 0;
}
```

코드8-3: 전단 변환

- 이때 입력 영상 일부가 잘리지 않도록 결과 영상 크기를 적절하게 확대함
- 가로 방향으로 층밀림을 수행하였으므로 src 영상과 dst 영상의 세로 크기는 동일하고, dst 영상의 가로 크기는 $\text{src.cols} + \text{src.rows} * \text{mx}$ 로 설정함

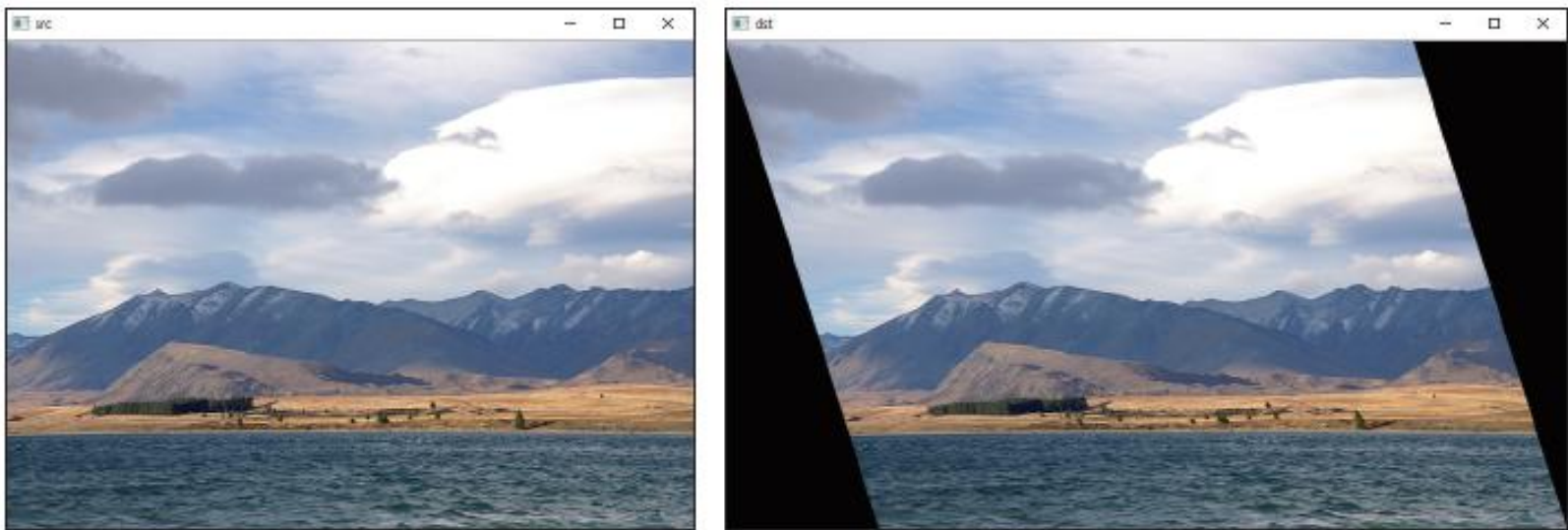


그림 8-7 영상의 전단 변환 예제 실행 결과

크기 변환

- 영상의 전체적인 크기를 확대 또는 축소하는 변환
- 그림 8-8에서 노란색 사각형 영역은 $w \times h$ 크기의 원본 영상이고, 녹색으로 표시한 사각형 영역은 $w' \times h'$ 크기로 확대된 결과 영상임

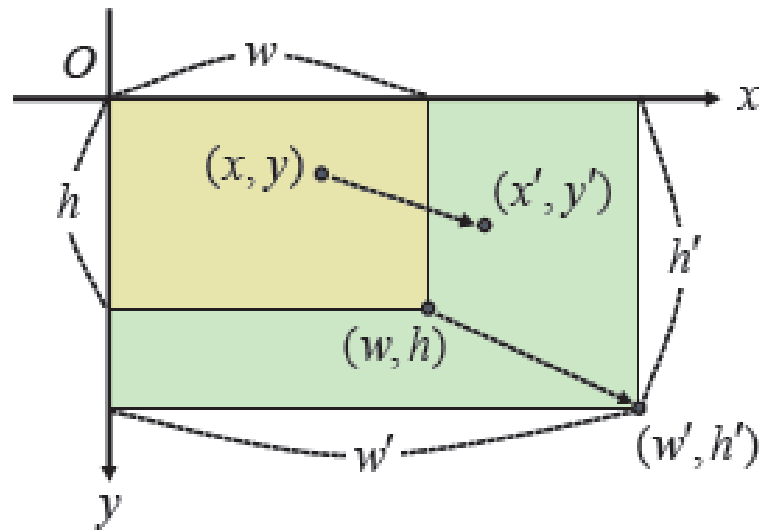


그림 8-8 영상의 크기 변환

크기 변환

- 원본 영상의 가로 픽셀 크기가 w 이고 결과 영상의 가로 크기가 w'
-> 가로 방향으로의 크기 변환 비율 $S_x = w'/w$
- y 방향으로의 크기 변환 비율 $S_y = h'/h$
- 입력 영상의 좌표 (x, y) 로부터 크기 변환 결과 영상의 좌표 (x', y') 를 계산하는 수식은 다음과 같음

$$\begin{cases} x' = s_x x \\ y' = s_y y \end{cases} \text{ 또는 } \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$= \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

크기 변환

- 앞 수식에서 s_x 또는 s_y 가 1보다 크면 영상이 확대되고, 1보다 작으면 축소됨
- 영상의 크기 변환을 나타내는 어파인 변환 행렬 M 은 다음과 같음

$$M = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \end{bmatrix}$$

- 영상을 크기 변환하려면 앞과 같은 형태의 2×3 실수 행렬 M 을 만들고, 이를 `warpAffine()` 함수 인자로 전달함

크기 변환

```
void resize(InputArray src, OutputArray dst,
            Size dsize, double fx = 0, double fy = 0,
            int interpolation = INTER_LINEAR);
```

- **src** 입력 영상
- **dst** 결과 영상
- **dsize** 결과 영상 크기
- **fx** x축 방향으로의 크기 변환 비율. dsize에 Size()를 지정한 경우에 사용됩니다.
- **fy** y축 방향으로의 크기 변환 비율. dsize에 Size()를 지정한 경우에 사용됩니다.
- **interpolation** 보간법 지정. INTER_NEAREST, INTER_LINEAR, INTER_CUBIC, INTER_AREA, INTER_LANCZOS4 중 하나를 지정합니다.

- 어파인 변환 행렬을 생성하고 warpAffine() 함수를 이용가능
- 영상의 크기를 변경하는 작업은 매우 빈번하게 사용되기 때문에 간단하게 크기를 변경할 수 있는 resize() 함수를 제공함

크기 변환

- 결과영상의 크기를 dsize를 통해 명시적으로 지정가능
- dsize에 0을 지정하는 경우 결과 영상의 크기는 다음과 같이 결정됨

$$\begin{cases} \text{dst.rows} = \text{round}(\text{src.rows} \times \text{fx}) \\ \text{dst.cols} = \text{round}(\text{src.cols} \times \text{fy}) \end{cases}$$

- 크기변환은 원점을 기준으로 영상이 확대, 축소됨
- `resize()` 함수의 여섯 번째 인자 `interpolation`에는 보간법(interpolation) 알고리즘을 나타내는 `InterpolationFlags` 열거형 상수를 지정함
- 보간법은 결과 영상의 픽셀 값을 결정하기 위해 입력 영상에서 주변 픽셀 값을 이용하는 방식을 의미함

크기 변환

- INTER_LINEAR 방법은 연산 속도가 빠르고 화질도 충분히 좋은 편이라서 널리 사용되고 있고, 기본값으로 지정되어 있음
- INTER_LINEAR 방법보다 더 좋은 화질을 원한다면 INTER_CUBIC 또는 INTER_LANCZOS4 상수를 사용하는 것이 좋음
- 영상을 축소하는 경우 INTER_AREA 방법을 사용하면 무아레(moiré) 현상이 적게 발생하며 화질 면에서 유리함

표 8-1 보간법 지정 InterpolationFlags 열거형 상수

InterpolationFlags 열거형 상수	설명
INTER_NEAREST	최근방 이웃 보간법
INTER_LINEAR	양선형 보간법
INTER_CUBIC	3차 보간법
INTER_AREA	픽셀 영역 리샘플링
INTER_LANCZOS4	8×8 이웃 픽셀을 사용하는 란초스(Lanczos) 보간법

코드8-4: 크기 변환

- 코드 8-4는 480×320 크기의 rose.bmp 장미 영상을 다양한 보간법으로 4배 확대하고, 결과 영상의 일부를 화면에 출력함

코드8-4: 크기 변환

```
// 생략
int main(void)
{
    Mat src = imread("rose.bmp");
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1; }
    Mat dst1, dst2, dst3, dst4;
    resize(src, dst1, Size(), 4, 4, INTER_NEAREST);
    resize(src, dst2, Size(1920, 1280));
    resize(src, dst3, Size(1920, 1280), 0, 0, INTER_CUBIC);
    resize(src, dst4, Size(1920, 1280), 0, 0, INTER_LANCZOS4);

    imshow("src", src);
    imshow("dst1", dst1(Rect(400, 500, 400, 400))); //91~92페이지 참고
    imshow("dst2", dst2(Rect(400, 500, 400, 400)));
    imshow("dst3", dst3(Rect(400, 500, 400, 400)));
    imshow("dst4", dst4(Rect(400, 500, 400, 400)));
    waitKey();
    return 0;
}
```

코드8-4: 크기 변환

- 그림 8-9에서 src는 입력 영상이고, dst1부터 dst4 영상은 `resize()` 함수에 의해 4배 확대된 결과 영상에서 추출한 부분 영상임

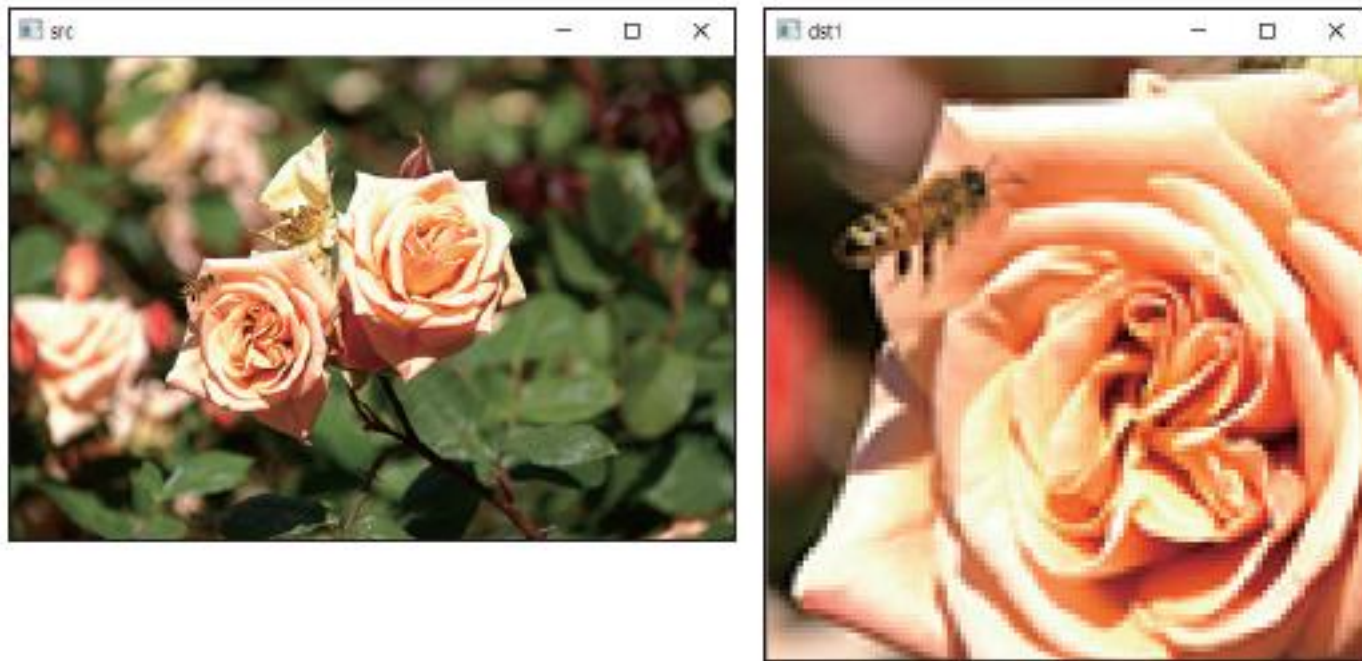


그림 8-9 다양한 보간법을 이용한 영상 확대 결과 비교

코드8-4: 크기 변환



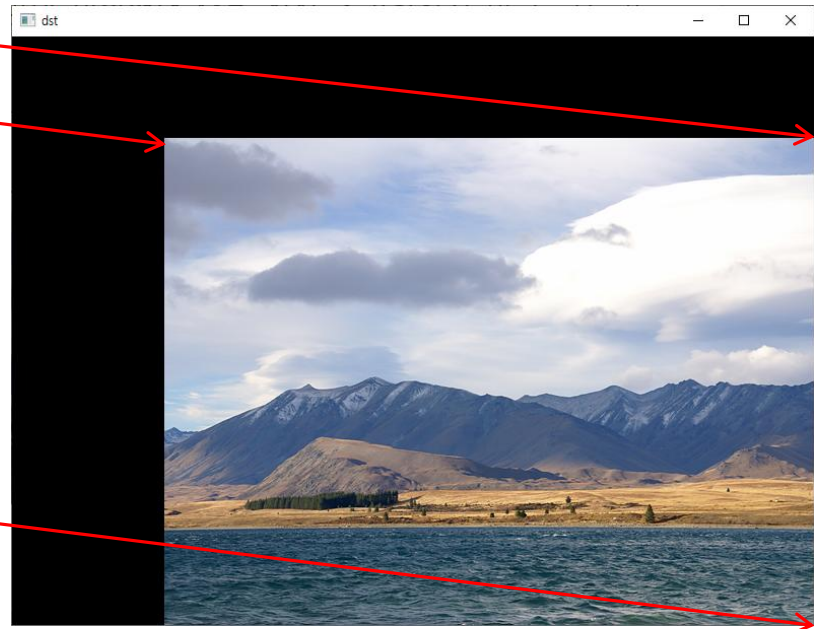
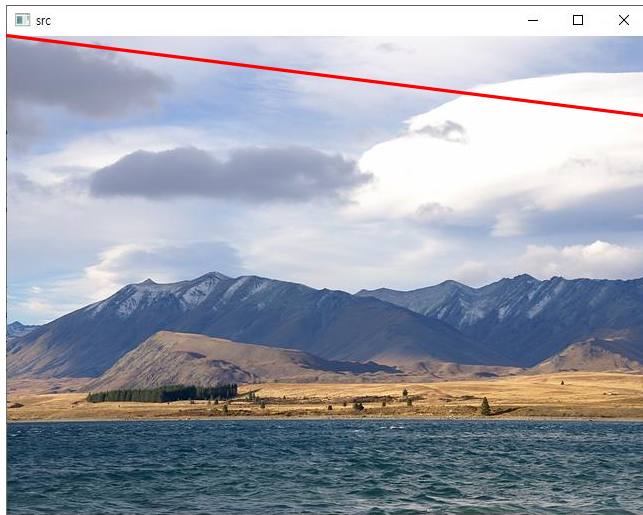
그림 8-9 다양한 보간법을 이용한 영상 확대 결과 비교

실습과제1

- 코드8-2에서 어파인 변환행렬 M 을 직접 초기화하지 말고 코드 8-1에서 처럼 `getAffineTransform`함수를 이용하여 구한후 화면에 출력하고 다음 처럼 `warpAffine`함수로 이동 변환하시오

```

c:\ D:\Users\W...  -  □  ×
[1, 0, 150;
 0, 1, 100]
  
```



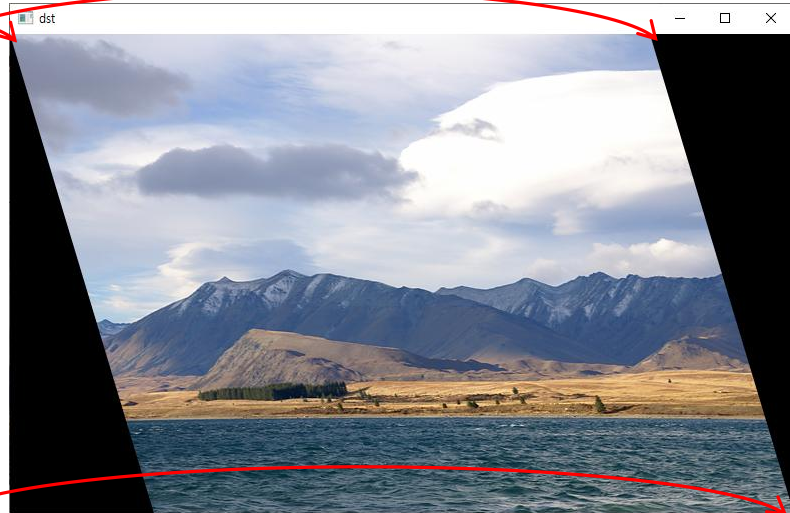
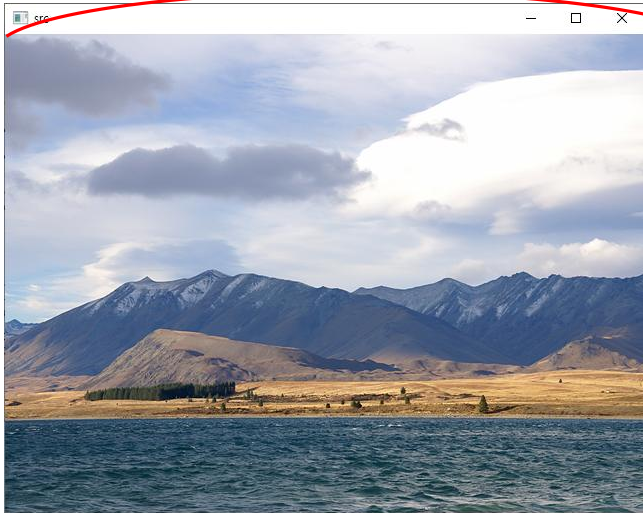
실습과제2

- 코드8-3에서 어파인 변환행렬 M 을 직접 초기화하지 말고 코드 8-1처럼 `getAffineTransform`함수를 이용하여 구한 후 화면에 출력하고 다음처럼 `warpAffine`함수로 이동 변환하시오

```

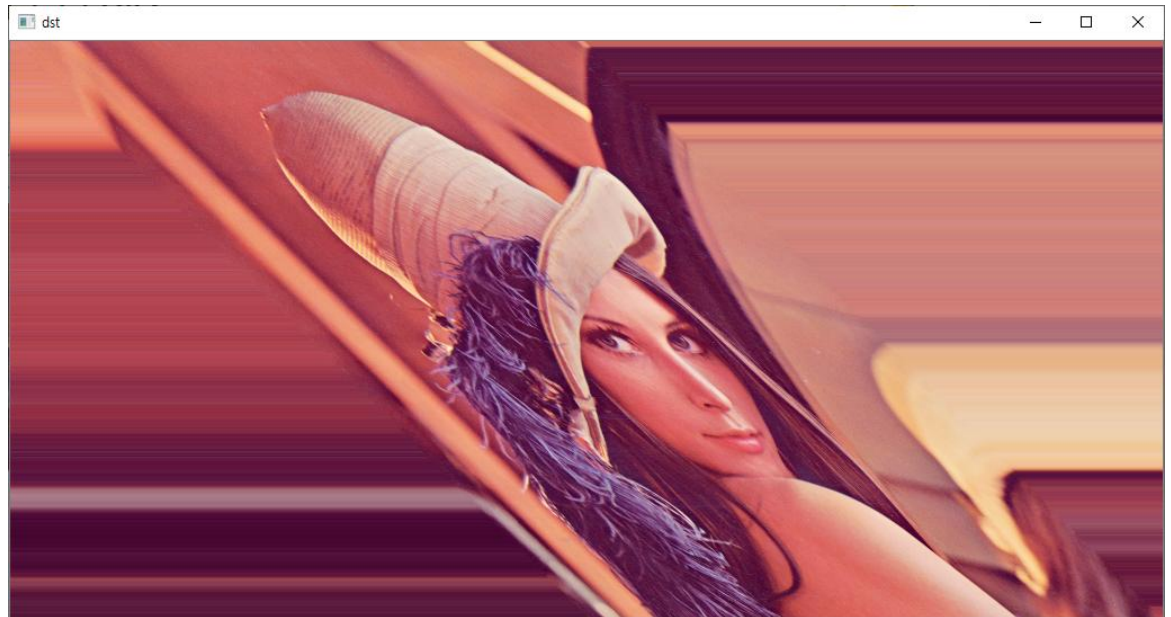
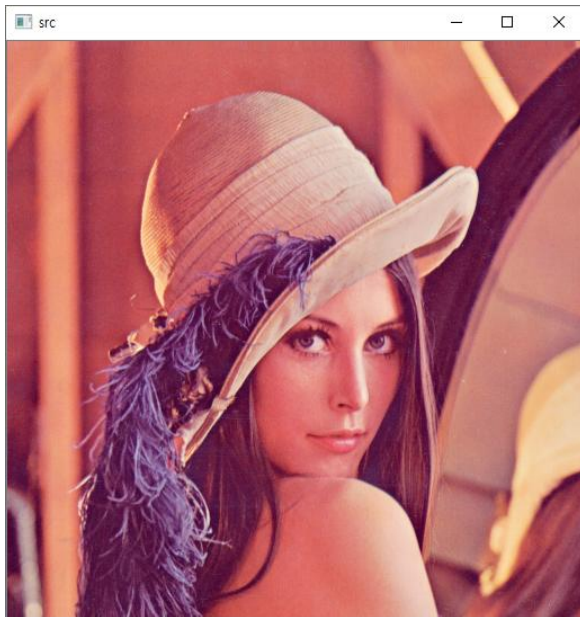
D:\Users\2sungryul\Dropbox...
[1, 0.3006263048016701, 0;
0, 1, 0]

```



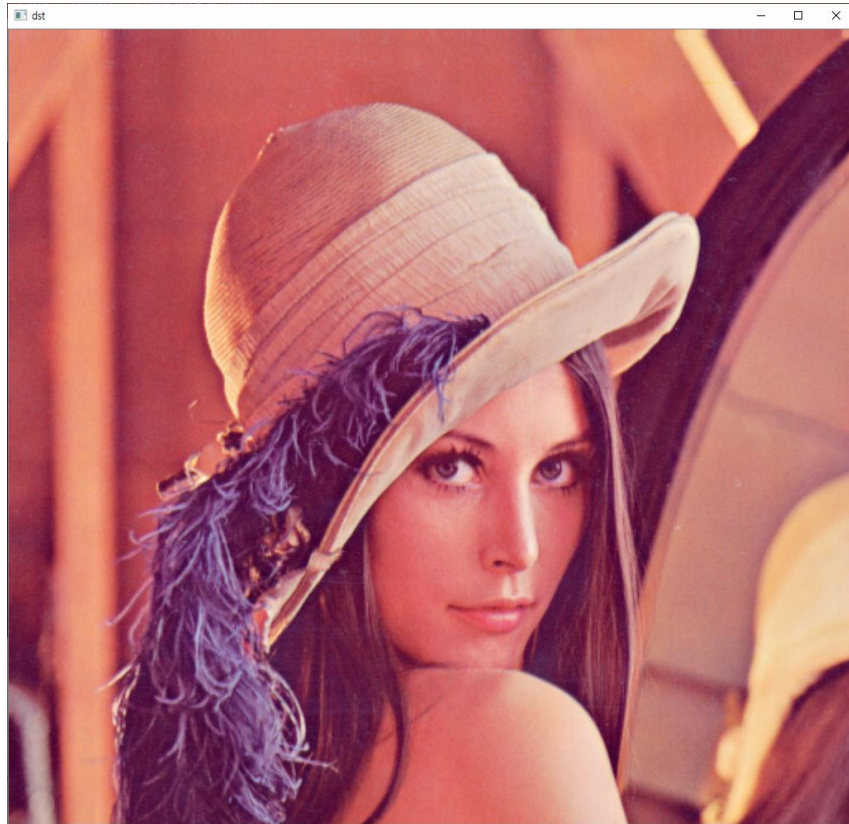
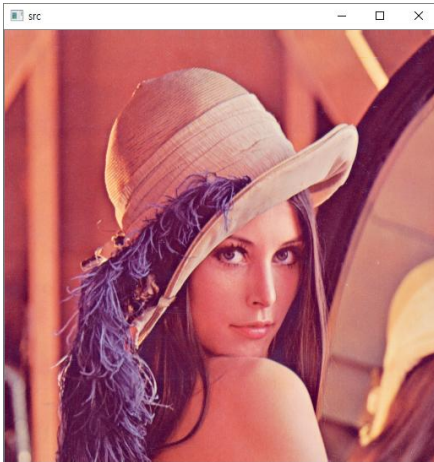
실습과제3

- X축방향으로 45도 기울어지도록 전단변환을 수행하고 검정색 영역을 원본의 가장자리의 픽셀값으로 채우시오. (aaaaa|abcdefg)
- BorderTypes 검색해볼 것(247페이지 참고할 것)



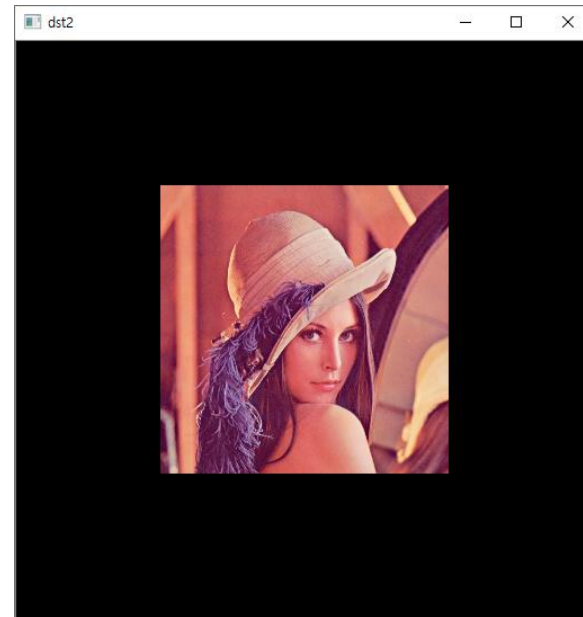
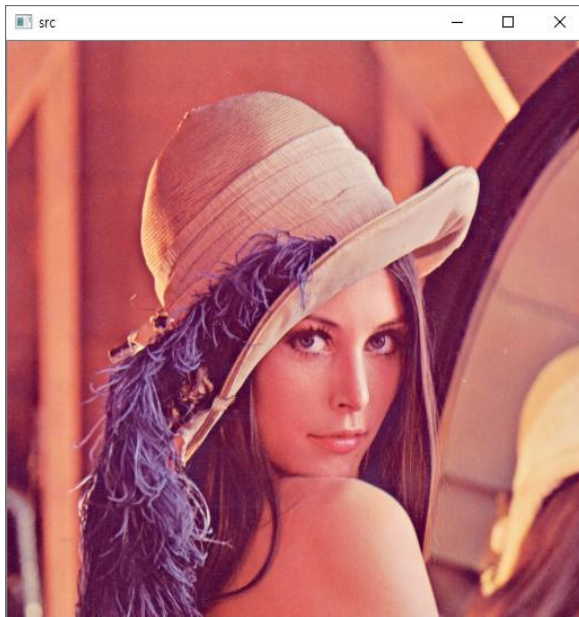
실습과제4

- 원본영상의 크기를 2배로 확대하는 어파인 변환을 `resize` 함수 이용하지 말고 코드 8-1처럼 `getAffineTransform` 함수를 이용하여 구한 후 화면에 출력하고 다음처럼 `warpAffine` 함수로 결과영상을 구하시오.



실습과제5

- 아래 그림처럼 결과가 나오도록 어파인 변환을 수행하시오. 영상의 크기는 $\frac{1}{2}$ 로 축소하고 결과창의 중앙에 위치하도록 하라.
- 힌트 : 크기변환과 이동변환을 모두 이용하라.



실습과제6

- 카메라로부터 입력 받은 동영상을 a를 누르면 원본 그대로 출력하고 b를 누르면 과제 5번처럼 처리하여 출력하시오. q를 누르면 종료하도록 하시오.
- 반복문안에 waitKey함수는 1번만 호출할 것, 2번 이상 호출하면 0점 처리

과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출 하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.